Snowflake PAT Cheatsheet

This cheatsheet provides step-by-step instructions for managing Programmatic Access Tokens (PAT) in Snowflake. Learn how to set up PATs, configure policies, and integrate with GitHub Actions for secure automation and API access.

Table of Contents

Prerequisites

Before you begin, ensure you have: - Snowflake account with appropriate permissions - Snowflake CLI installed and configured - GitHub CLI installed and configured - jq installed for JSON parsing

> [!IMPORTANT]
> Verify your Snowflake CLI connection before proceeding.
> See the Snowflake CLI connection guide if needed.

Quick Start

1. Environment Setup

```
# Set required environment variables
export SNOWFLAKE_USER="your Snowflake username"
export SNOWFLAKE_DEFAULT_CONNECTION_NAME="your Snowflake
        connection name from ~/.snowflake/config.toml"
export MY_PAT_DEMO_ROLE="the role you want to restrict the PAT"
```

Example configuration:

```
export SNOWFLAKE_USER="demo"
export SNOWFLAKE_DEFAULT_CONNECTION_NAME="trial"
export MY_PAT_DEMO_ROLE="public"
```

2. Verify Connection

```
snow connection test --format=json
```

3. Create Required Objects

```
-- Create database and schemas
CREATE DATABASE my_demo_db;
CREATE SCHEMA my_demo_db.policies;
CREATE SCHEMA my_demo_db.networks;

-- Create authentication policy
CREATE AUTHENTICATION POLICY my_demo_db.policies.demos_auth_policy
  AUTHENTICATION_METHODS = ('PASSWORD', 'OAUTH', 'KEYPAIR',
        'PROGRAMMATIC_ACCESS_TOKEN')
  PAT_POLICY = (
    DEFAULT_EXPIRY_IN_DAYS=7,
    MAX_EXPIRY_IN_DAYS=90,
    NETWORK_POLICY_EVALUATION = ENFORCED_NOT_REQUIRED
  );
```

4. Configure Network Access

```
# Get GitHub Actions IP ranges (IPv4 only)
GH_CIDRS=$(curl -s https://api.github.com/meta | jq -r
        '.actions | map(select(contains(":") | not)) | map("'\''"
        + . + "'\''") | join(",")')

# Get local IP
LOCAL_IP="$(dig +short myip.opendns.com @resolver1.opendns.com)/
        32"
```

```
# Create variables for SQL
export CIDR_VALUE_LIST="$GH_CIDRS,$LOCAL_IP"

-- Create network rule
CREATE NETWORK RULE
        my_demo_db.networks.pat_gh_actions_local_access_rule
  MODE = INGRESS
  TYPE = IPV4
  VALUE_LIST = ($CIDR_VALUE_LIST)
  COMMENT = 'Allow only GitHub and local machine IPv4 addresses';

-- Create network policy
CREATE NETWORK POLICY ALLOW_ALL_PAT_NETWORK_POLICY
  ALLOWED_NETWORK_RULE_LIST =
        ('my_demo_db.networks.pat_gh_actions_local_access_rule');

-- Apply network policy to user
ALTER USER $SNOWFLAKE_USER SET
        NETWORK_POLICY='ALLOW_ALL_PAT_NETWORK_POLICY';
```

5. Generate and Test PAT

```
# Generate PAT
export SNOWFLAKE_PASSWORD=$(snow sql --query "ALTER USER IF
        EXISTS $SNOWFLAKE_USER ADD PAT my_demo_pat
        ROLE_RESTRICTION = $MY_PAT_DEMO_ROLE" --format=json | jq -
        r '.[] | .token_secret')

# Get connection details
snow connection test --format=json > connection.json
export SNOWFLAKE_ACCOUNT=$(jq -r '.Account' connection.json)
export SNOWFLAKE_HOST=$(jq -r '.Host' connection.json)

# Test PAT connection
snow sql --temporary-connection \
  --account="${SNOWFLAKE_ACCOUNT}" \
  --host="${SNOWFLAKE_HOST}" \
  --user="${SNOWFLAKE_USER}" \
  --query "select current_user(), current_role()"
```

6. GitHub Actions Integration

```
# Set GitHub Secrets
gh secret set SNOWFLAKE_PASSWORD --body "$SNOWFLAKE_PASSWORD"
gh secret set SNOWFLAKE_ACCOUNT --body "$SNOWFLAKE_ACCOUNT"
gh secret set SNOWFLAKE_USER --body "$SNOWFLAKE_USER"
gh secret set SNOWFLAKE_HOST --body "$SNOWFLAKE_HOST"
```

Cleanup

1. Remove PAT

```
snow sql --query "ALTER USER IF EXISTS $SNOWFLAKE_USER REMOVE PAT
        my_demo_pat"
```

2. Remove Network Configuration

```
-- Remove network policy from user
ALTER USER $SNOWFLAKE_USER UNSET NETWORK_POLICY;

-- Drop network rule and policy
DROP NETWORK RULE
        my_demo_db.networks.pat_gh_actions_local_access_rule;
DROP NETWORK POLICY ALLOW_ALL_PAT_NETWORK_POLICY;
```

3. Remove GitHub Secrets

```
gh secret delete SNOWFLAKE_PASSWORD
gh secret delete SNOWFLAKE_ACCOUNT
gh secret delete SNOWFLAKE_USER
gh secret delete SNOWFLAKE_HOST
```

Security Best Practices

Authentication

- Use role restrictions with PATs to limit scope
- Set appropriate expiration periods (recommended: 7-30 days)
- Rotate PATs regularly and before expiration

Network Security

- Always use HTTPS for API calls
- Restrict network access to specific IP ranges in production
- Regularly audit and update allowed IP ranges

Credential Management

- Store PATs securely using secret managers
- Never commit PATs to version control
- Use separate PATs for different applications/services

Reference Links

- Snowflake PAT Documentation
- Snowflake Authentication Policies
- Snowflake Network Policies
- Snowflake Network Rules
- GitHub Actions
- About GitHub’s IP addresses
- Snowflake CLI Documentation
- Snowflake PAT Demo